

Computer Architecture: *Pipelines*

Dr. Colin O'Flynn

Lectures

- How does a program run on a machine?
- How do you compile from high-level code to machine code?
- How does the machine run? (Control Flow)
- How does the machine store, access, and control data?
- How does the machine execute instructions?
- **How is execution made more efficient?**
- *How do we measure execution?*

Pipeline Context



**JADE COMPUTER
SUPER-386**

Configuration	Price
20 MHz	\$1498
25 MHz	\$1598
25 MHz CACHE	\$1998
33 MHz Cache	\$2398

Full Featured Professional Systems

- True 20, 25 or 33 MHz 80386 CPU
- 1 MB or 32 BIT RAM Expands to 6 MB
- 384K Shadow RAM
- 1.2 MB 5¼" or 1.44 MB 3½" Floppy Disk Drive
- Fast 1:1 Interleave Dual Hard Disk/ Dual Floppy Disk Controller
- 80387 Coprocessor Socket
- Full Size Case with 5 Half Height Drive Bays
- 101 Key Enhanced Keyboard
- 200 Watt Power Supply
- Built-in Clock/Calendar

RISC-V Instruction Set

RV32I Base Integer Instruction Set

Format	Name	Pseudocode
<code>LUI rd,imm</code>	Load Upper Immediate	$rd \leftarrow \text{imm}$
<code>ADD rd,rs1,rs2</code>	Add	$rd \leftarrow \text{sx}(\text{rs1}) + \text{sx}(\text{rs2})$
<code>SUB rd,rs1,rs2</code>	Subtract	$rd \leftarrow \text{sx}(\text{rs1}) - \text{sx}(\text{rs2})$
<code>SLL rd,rs1,rs2</code>	Shift Left Logical	$rd \leftarrow \text{ux}(\text{rs1}) \ll \text{rs2}$
<code>SLT rd,rs1,rs2</code>	Set Less Than	$rd \leftarrow \text{sx}(\text{rs1}) < \text{sx}(\text{rs2})$

A Simple RISC-V Program

C (High Level)

```
int AddAndMul(int a, int b){  
    int c;  
    c = a + b;  
    c = c * a;  
    c = c * b;  
    return c;  
}
```

Compiler

ASM (Lower Level)

```
AddAndMul(int, int):  
    add    a5, a0, a1  
    mul    a0, a5, a0  
    mul    a0, a1, a0  
    ret
```

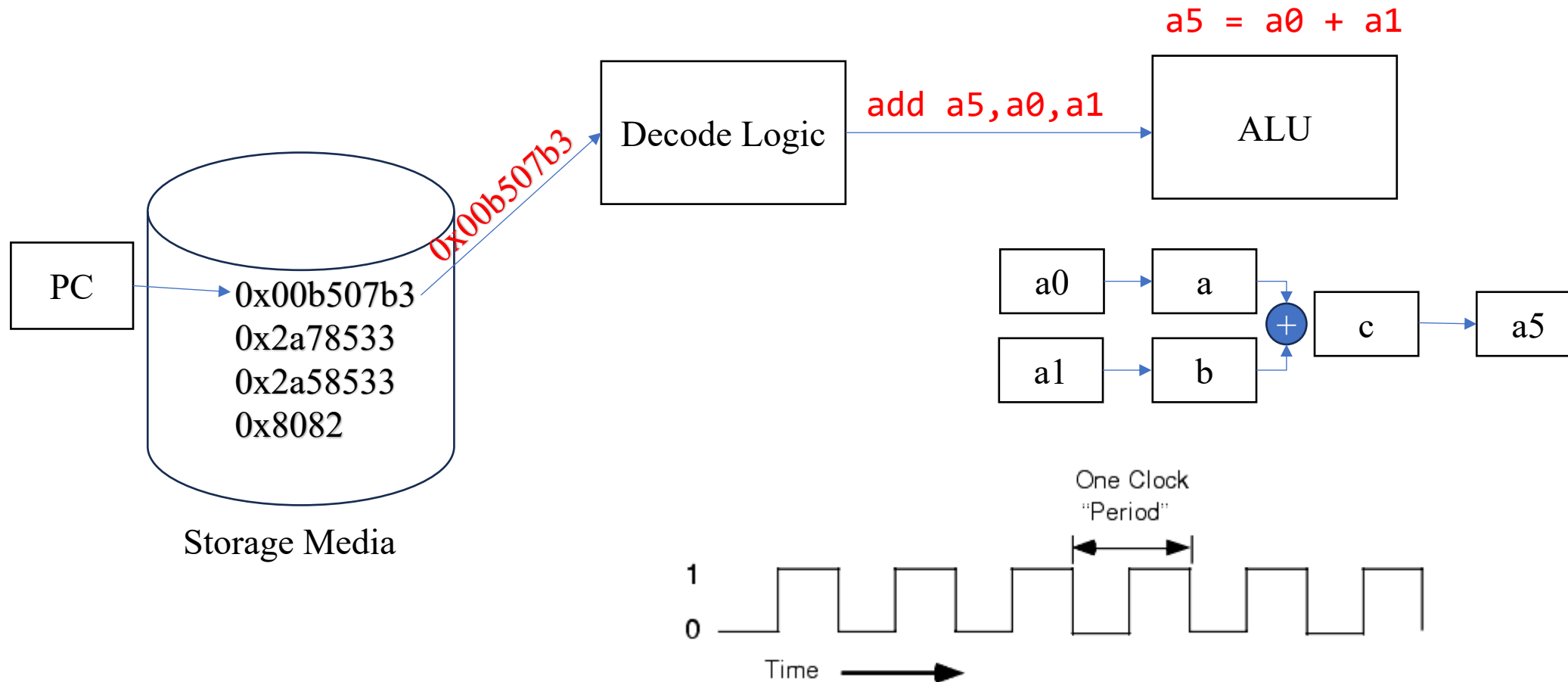
Assembler

Binary / Machine

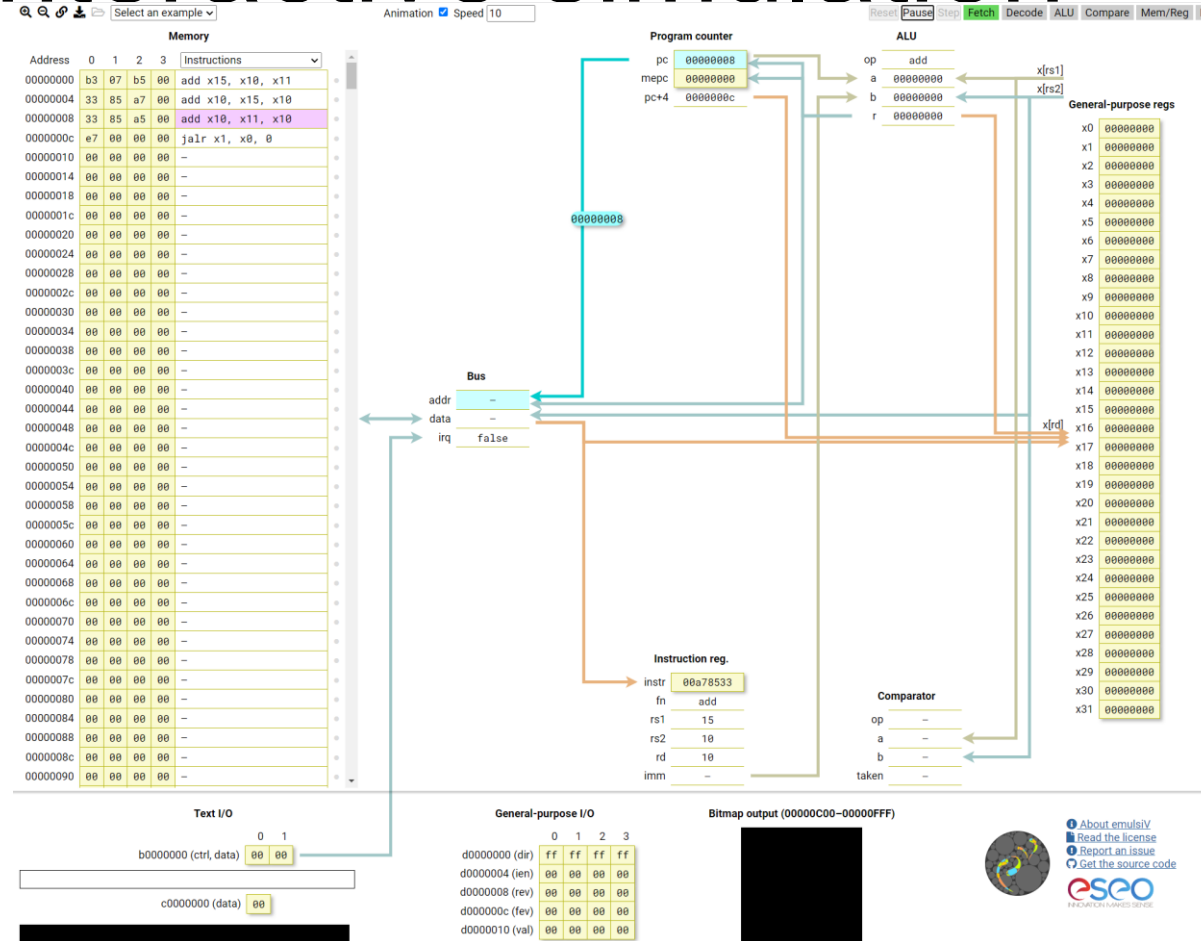
```
0x00b507b3  
0x2a78533  
0x2a58533  
0x8082
```

*NOTE: in RISC-V **a0/a1**/etc are general-purpose registers.*

Executing a Simple Program (Naïve)



e.g., interactive simulation

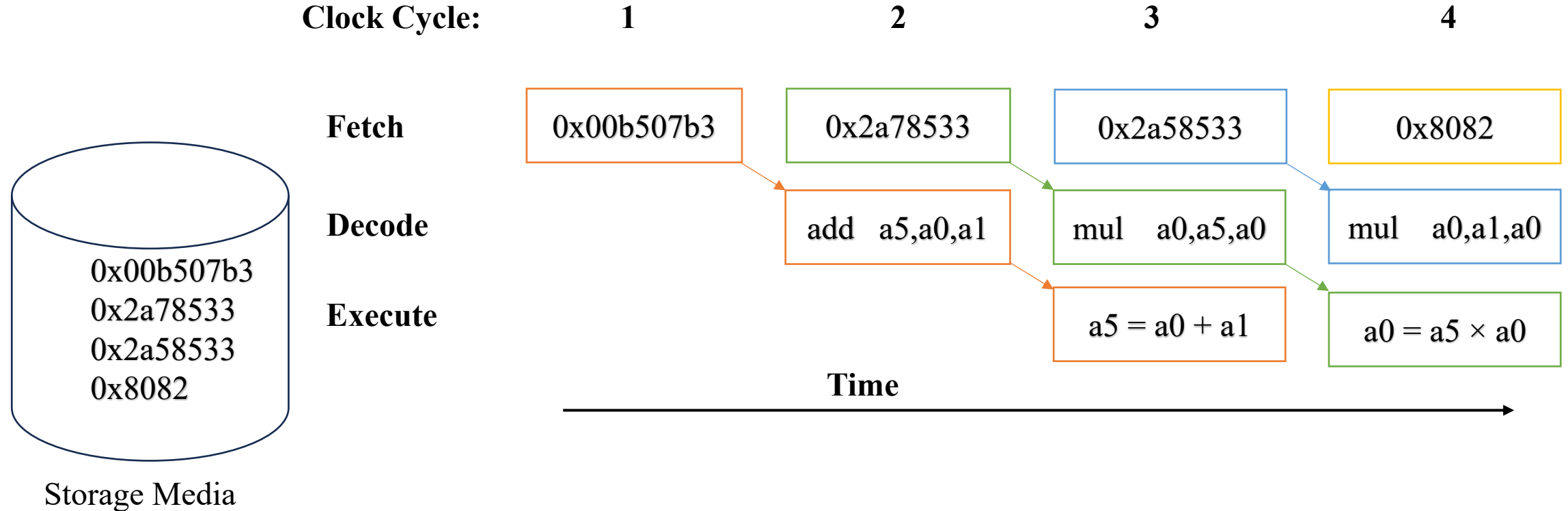


<https://eseo-tech.github.io/emulsiV/>

```
add x15, x10, x11
add x10, x15, x10
add x10, x11, x10
jair x1, x0, 0
```

About Logic Propagation Time

Simple Pipeline Example



*NOTE: in RISC-V **a0/a1**/etc are general-purpose registers.*

Human Pipelining Examples

You probably already “pipeline” lots of tasks, e.g.:

- Doing laundry
- Cooking / Baking
- Making instant noodles



Fill and Turn On
Kettle



Open Ramen &
Packets / Grab Utensils



Pour Water



Wait



Eat

2 vs 3 vs 5 vs ... stage pipeline

Frontend

Plans the code the backend will execute. Goal is to keep the backend busy as much as possible.

(1) Instruction Fetch & Decode

(1) Instruction Fetch (2) Decode

(1) Instruction Fetch (2) Decode

(1) Instruction Fetch (2) Decode

Backend

Actually executes the instructions, including writing anything to memory or registers.

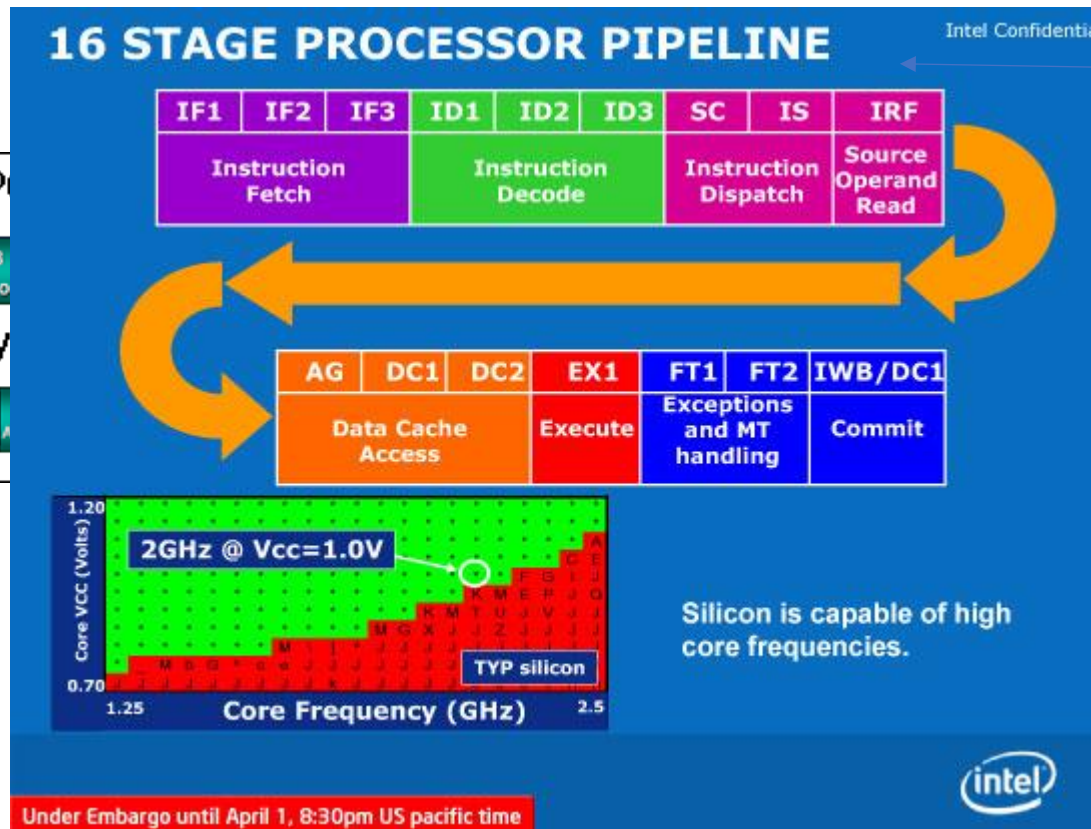
(2) Execute & Writeback

(3) Execute & Writeback

(3) Execute (4) Writeback

(3) Execute (4) Memory (5) Register Writeback

Substages within Stages



Intel Atom (2008)

Atom 4 had 20 stage pipeline(2000). A later version (P4E, 2004) went up to 31. Things have changed since then.

Pipeline Hazards

Control hazard

- Need result of a conditional (branch?) before conditional result is ready

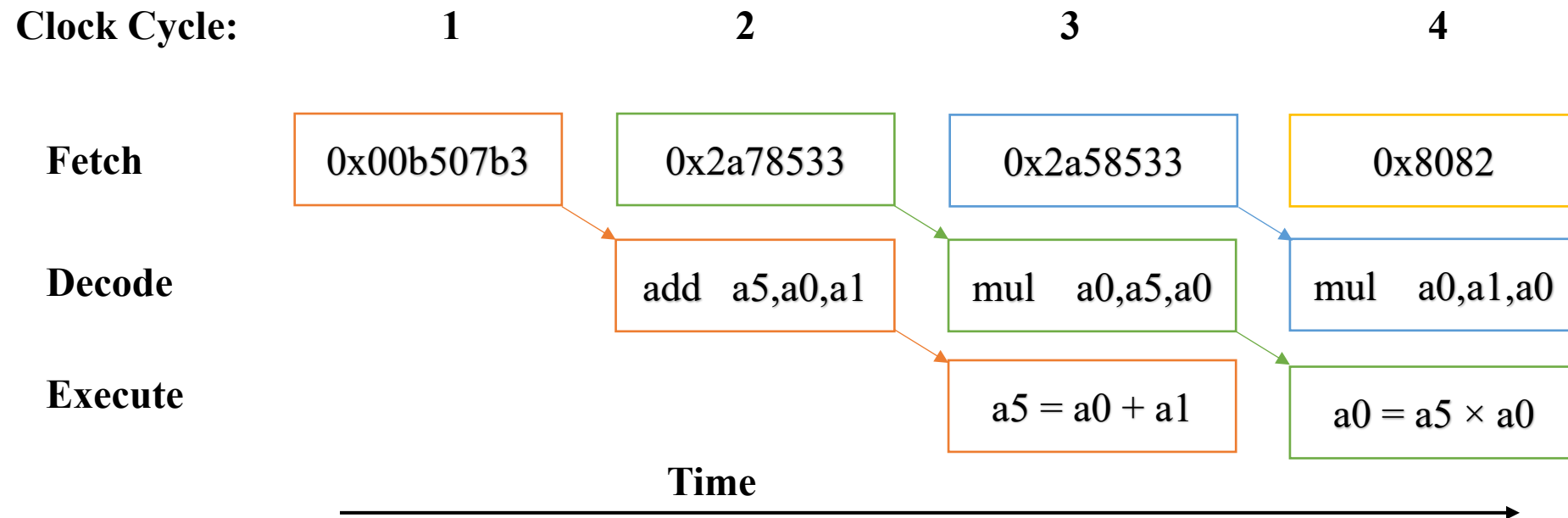
Structural hazard

- Need the same resource (e.g., multiplier hardware)

Data hazard

- Instruction to execute needs result of not-finished instruction

What Hazards are Here?



Remember – Logic can be SLOW

Example – NEORV32 Multiply Timing

3.7.5. M ISA Extension

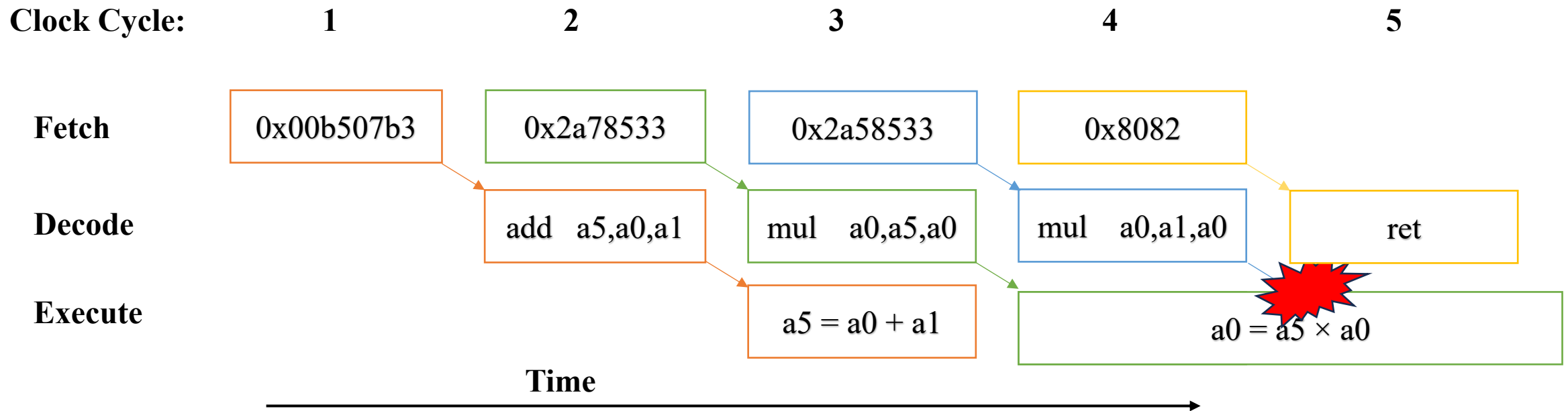
Hardware-accelerated integer multiplication and division operations are available via the RISC-V M ISA extension.

Table 46. Instructions and Timing

Class	Instructions	Execution cycles
Multiplication	<code>mul</code> <code>mulh</code> <code>mulhsu</code> <code>mulhu</code>	36; FAST_MUL: 4
Division	<code>div</code> <code>divu</code> <code>rem</code> <code>remu</code>	36

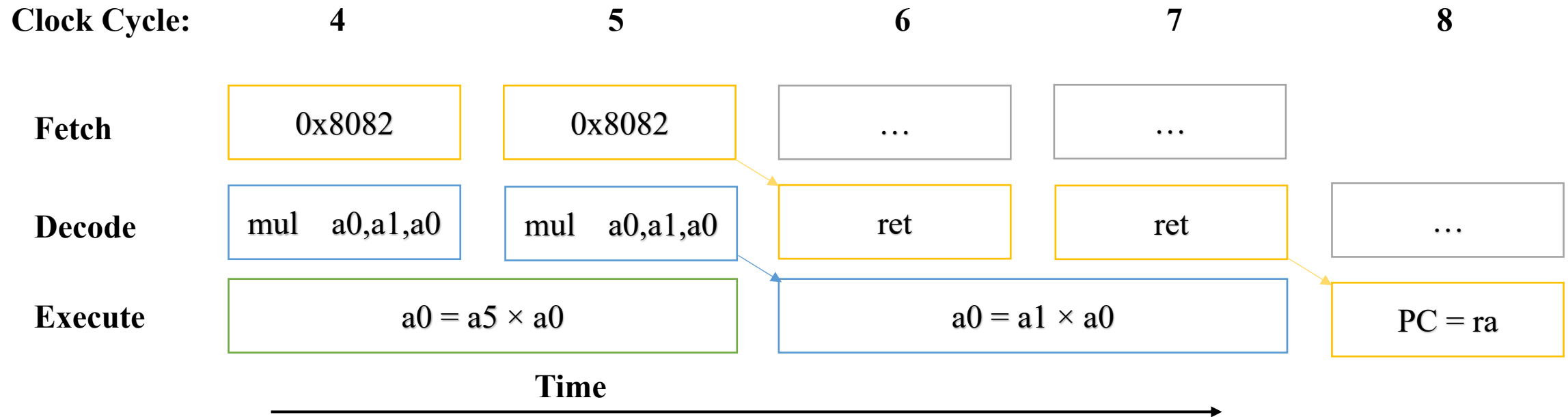
<https://stnolting.github.io/neorv32/>

Example: 2-Cycle Multiply



 Here is **BOTH** a structural hazard & a data hazard (if we had another multiplier)

Example: 2-Cycle Multiply



*NOTE: in RISC-V **a0/a1**/etc are general-purpose registers. **ra** is “Return Address” register (typically ‘**Link Register**’ in Arm).*

Pipeline Stalls

Stall:

- Pause the pipeline while waiting for availability of resource (e.g., the slow multiply).

Example: Branch Instruction

C (High Level)

```
int comp(int a, int b){  
    if (a > b){  
        return a+b;  
    } else {  
        return a-b;  
    }  
}
```

Compiler

ASM (Lower Level)

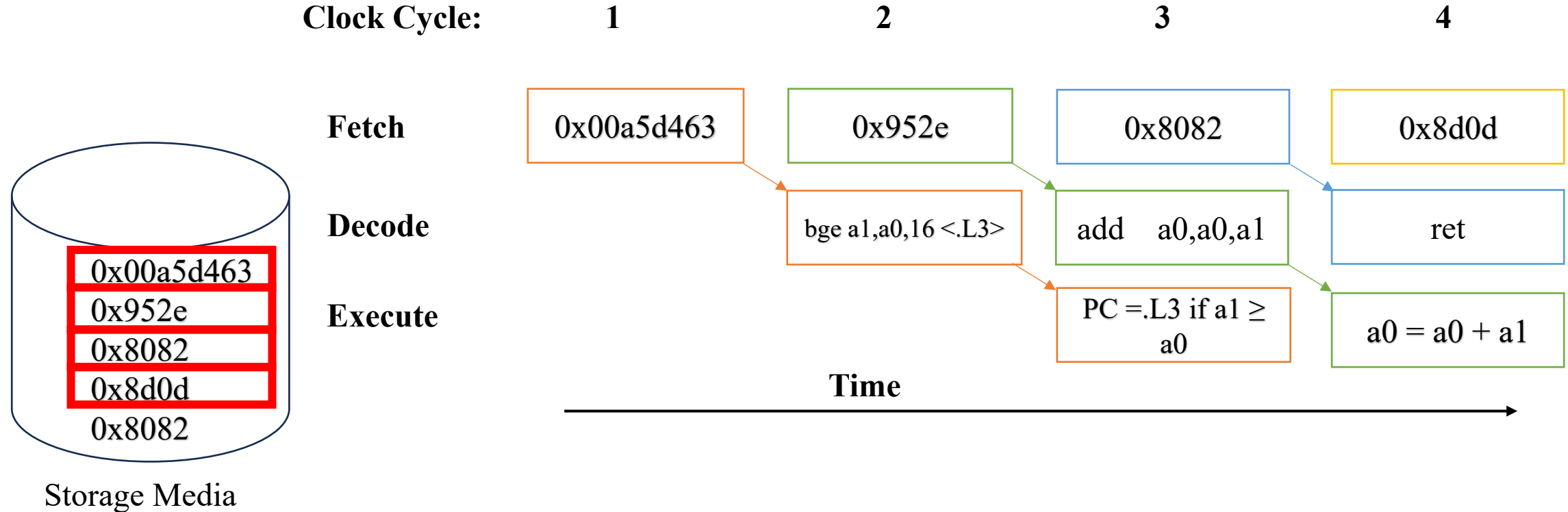
```
comp(int, int):  
    bge     a1,a0,16 <.L3>  
    add     a0,a0,a1  
    ret  
.L3:  
    sub     a0,a0,a1  
    ret
```

Assembler

Binary / Machine

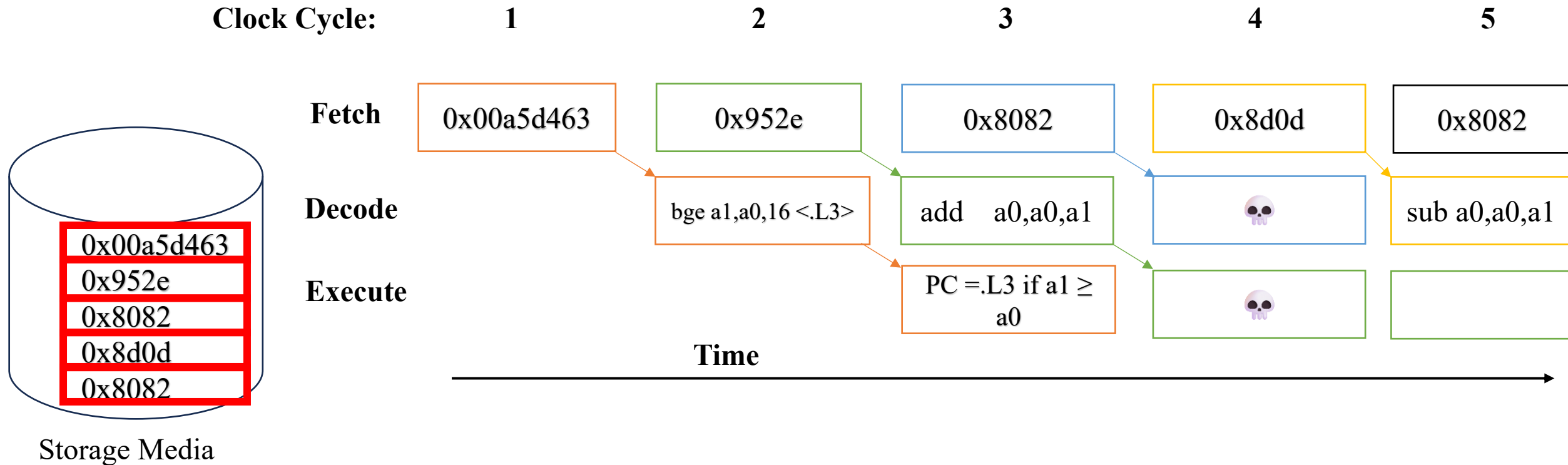
```
0x00a5d463  
0x952e  
0x8082  
0x8d0d  
0x8082
```

Pipeline with Branch



*NOTE: in RISC-V **a0/a1**/etc are general-purpose registers.*

Pipeline with Branch



*NOTE: in RISC-V **a0/a1**/etc are general-purpose registers.*

Pipeline Flush

Flush:

- Control flow change means pipeline had the wrong instructions – need to throw away the current contents.

e.g. Typical Specifications – Instruction Latency

3.7.1. I ISA Extension

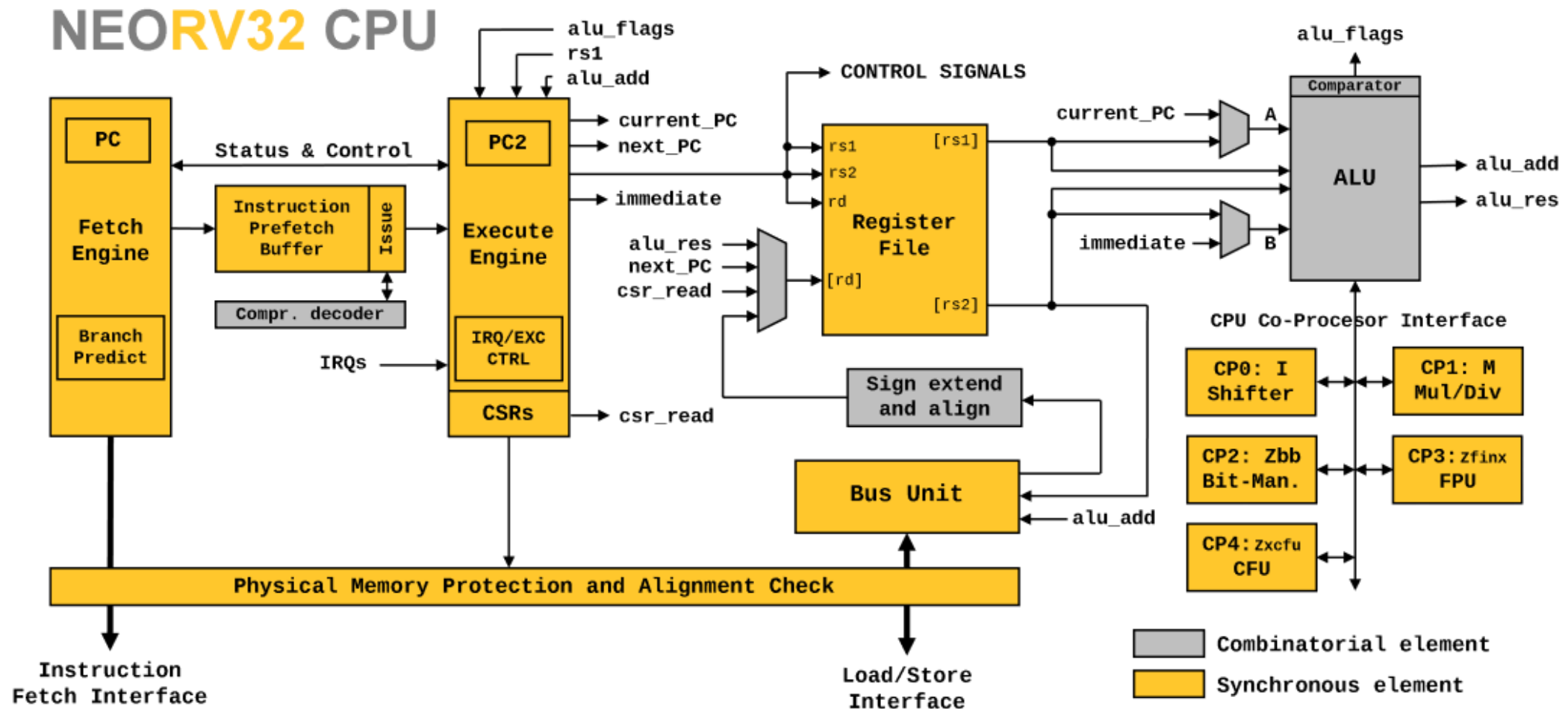
The I ISA extensions is the base RISC-V integer ISA that is always enabled.

Table 43. Instructions and Timing

Class	Instructions	Execution cycles
ALU	add[i] slt[i] slt[i]u xor[i] or[i] and[i] sub lui auipc	2
ALU shifts	sll[i] srl[i] sra[i]	3 + 1..32; FAST_SHIFT: 4
Branches	beq bne blt bge bltu bgeu	taken: 6; not taken: 3
Jump/call	jal[r]	6
Load/store	lb lh lw lbu lhu sb sh sw	4
System	ecall ebreak	3
Data fence	fence	5
Illegal inst.	-	3

<https://stnolting.github.io/neorv32/>

Example: A Simple CPU Architecture



Branch / Pipeline Stall

Q: In previous example – why not just stall pipeline until result is known?

A: Don't know about the branch until “later” in pipeline.

A: It's more efficient to just assume branch isn't taken & keep pipeline up. If branch is random you are right 50% of the time, rather than stalling 100% of the time.

Branch Prediction

What if we could *predict* which branches that will be taken? Could get better than 50%! Don't need time travel even...



Motivation for Branch Prediction

```
for(int i = 0; i < datalen; i++){  
    process_byte(data[i]);  
}
```

```
loop(char*, int):  
    addi    sp, sp, -16  
    sw      s0, 8(sp)  
    sw      s1, 4(sp)  
    sw      s2, 0(sp)  
    sw      ra, 12(sp)  
    mv      s2, a0  
    mv      s1, a1  
    li      s0, 0  
  
    .L6:  
    blt     s0, s1, .L7  
    lw      ra, 12(sp)  
    lw      s0, 8(sp)  
    lw      s1, 4(sp)  
    lw      s2, 0(sp)  
    addi    sp, sp, 16  
    jr      ra  
  
    .L7:  
    add     a5, s2, s0  
    lbu     a0, 0(a5)  
    addi    s0, s0, 1  
    call    process_byte(char)  
    j       .L6
```

Motivation for Branch Prediction

In this loop example → branch will be taken continuously until only the *final* iteration. Imagine if `datalen = 512`, branch not taken 0.195% of time!

The default pipeline behaviour of “assume branch not taken” will have a large (negative) impact on this code-path!

e.g., for NEORV32, adds +3 *additional* cycles to the loop iteration each time it's wrong.

Branch Prediction Example: Pentium

Branch prediction logic: holds some statement information about latest N seen branch instructions. This state indicates the likely output of the branch:

History Bits	Resulting Description	Prediction made	If branch taken	If branch not taken
11	Strongly Taken	Branch Taken	Remains in same state	Downgraded to weakly taken
10	Weakly Taken	Branch Taken	Upgraded to strongly taken	Downgraded to weakly not taken
01	Weakly Not Taken	Branch Not Taken	Upgraded to weakly taken	Downgraded to strongly not taken
00	Strongly Not Taken	Branch Not Taken	Upgraded to weakly not taken	Remains in same state

<https://www.geeksforgeeks.org/branch-prediction-in-pentium/>

Branch Prediction to Speculative Execution

- Systems may *speculatively execute* instructions once we have branch prediction.
- Based on predicted branch, load new data into pipeline, and start executing it if something is slow (e.g., waiting on memory).
- Need to “unwind” any executed instructions if branch wasn’t taken.
 - e.g., can’t start writing to registers yet!

Out of Order Execution

```
int ooo(int a, int b, int c){  
    int e = a * b + 14;  
    int f = a + b + c;  
    return e * f;  
}
```



```
ooo(int, int, int):  
    mul    a5,a0,a1  
    addi   a5,a5,14  
    add    a0,a0,a1  
    add    a0,a0,a2  
    mul    a0,a5,a0  
    ret
```

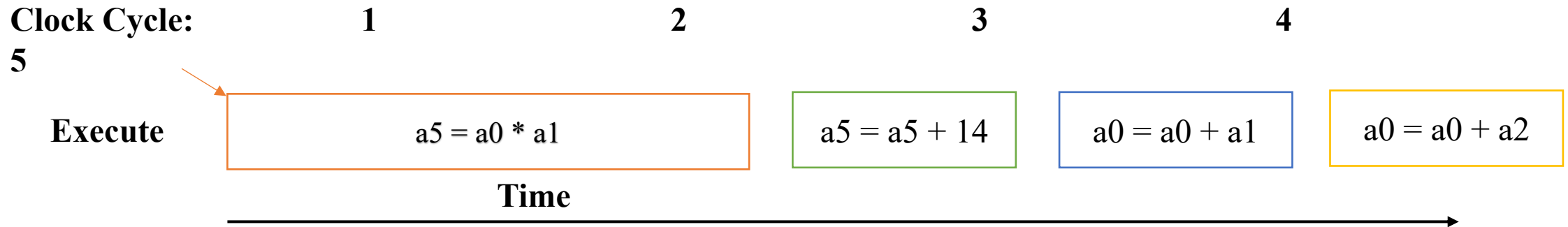
Out of Order Execution

```
int ooo(int a, int b, int c){  
    int e = a * b + 14;  
    int f = a + b + c;  
    return e * f;  
}
```



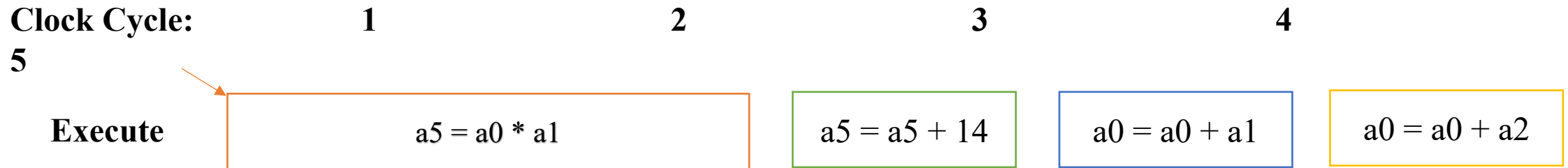
```
ooo(int, int, int):  
    mul    a5,a0,a1  
    addi   a5,a5,14  
    add    a0,a0,a1  
    add    a0,a0,a2  
    mul    a0,a5,a0  
    ret
```

Out of Order Execution



```
ooo(int, int, int):  
    mul    a5, a0, a1  
    addi   a5, a5, 14  
    add    a0, a0, a1  
    add    a0, a0, a2  
    mul    a0, a5, a0  
    ret
```

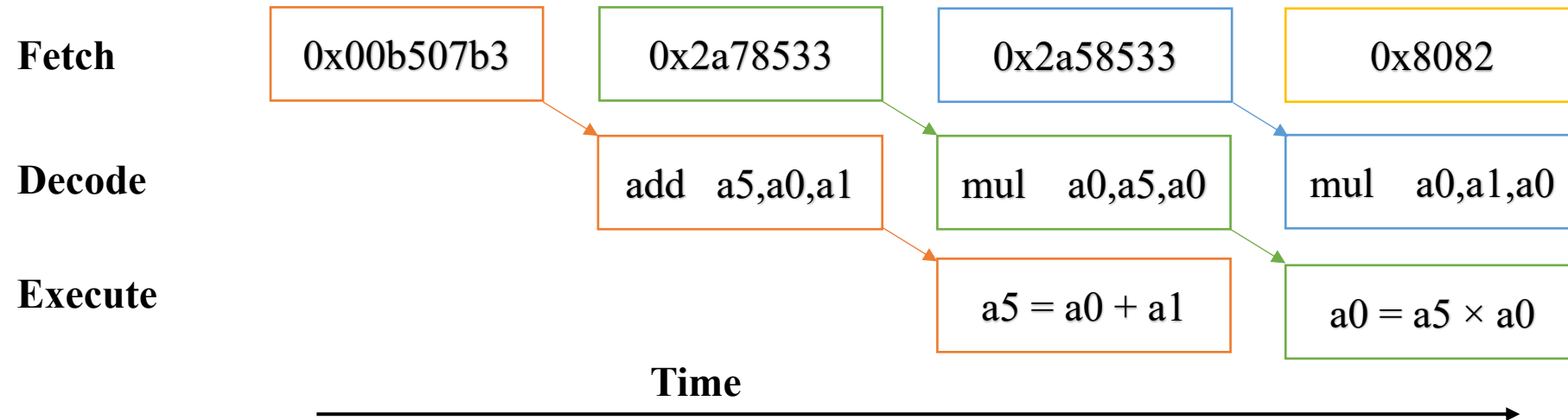
Out of Order Execution



Time

```
ooo(int, int, int):  
    mul    a5, a0, a1  
    addi   a5, a5, 14  
    add    a0, a0, a1  
    add    a0, a0, a2  
    mul    a0, a5, a0  
    ret
```

Interrupts



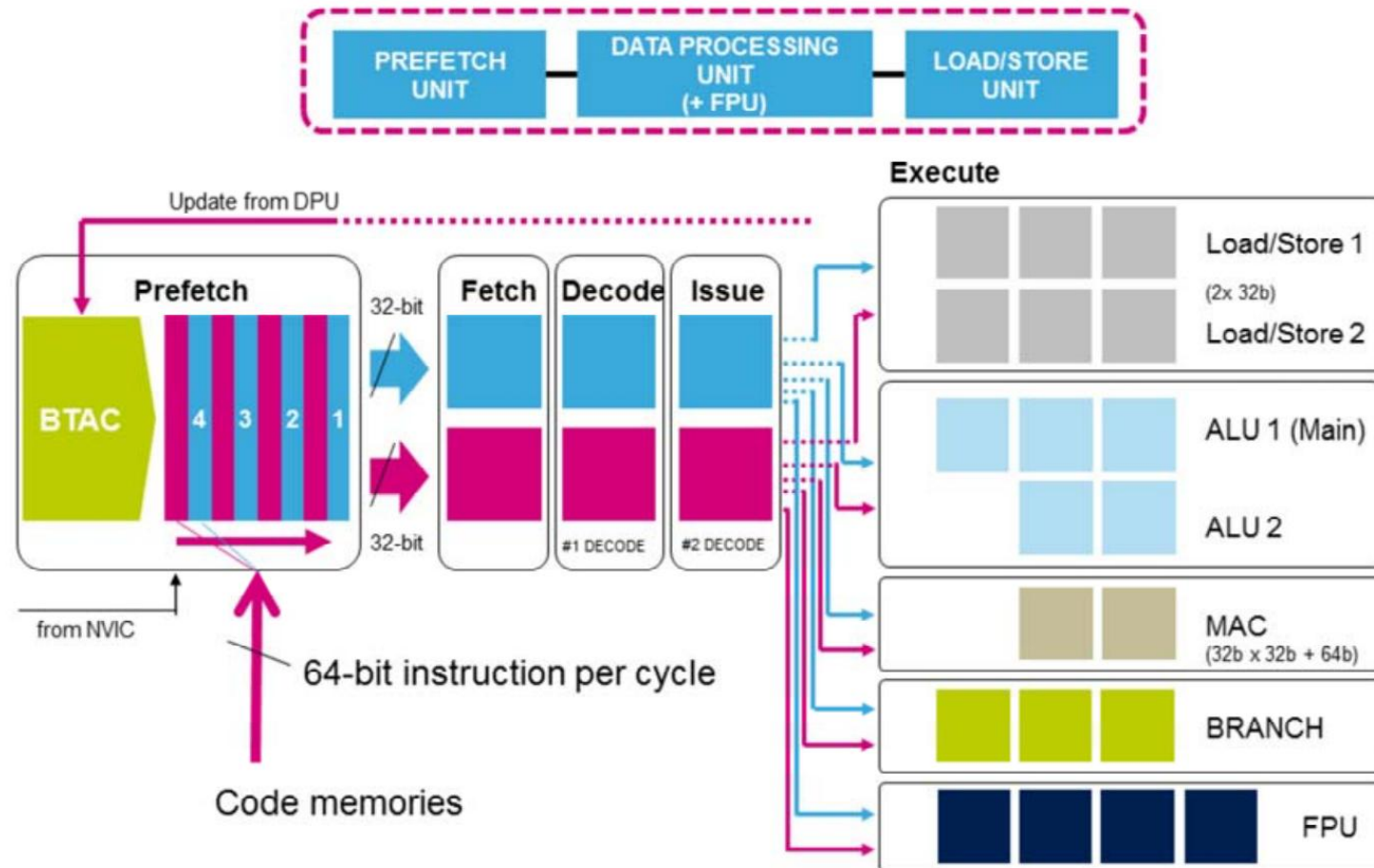
Cortex-M Pipeline Comparisons

Arm Core	Cortex M0 ^[18]	Cortex M0+ ^[19]	Cortex M1 ^[20]	Cortex M3 ^[21]	Cortex M4 ^[22]	Cortex M7 ^[23]	Cortex M23 ^[24]	Cortex M33 ^[25]
ARM architecture	ARMv6-M ^[15]	ARMv6-M ^[15]	ARMv6-M ^[15]	ARMv7-M ^[16]	ARMv7E-M ^[16]	ARMv7E-M ^[16]	ARMv8-M Baseline ^[31]	ARMv8-M Mainline ^[31]
Computer architecture	Von Neumann	Von Neumann	Von Neumann	Harvard	Harvard	Harvard	Von Neumann	Harvard
Instruction pipeline	3 stages	2 stages	3 stages	3 stages	3 stages	6 stages	2 stages	3 stages

https://en.wikipedia.org/wiki/ARM_Cortex-M

6-Stage Pipeline?

ARM Cortex-M7 → Dual-issue



Pipeline Summary

- **Pipelining** is a *natural* way to improve throughput.
- Pipelines are typically 2~5 stages in most microcontrollers you'll use.
 - Stages split between Frontend & Backend
 - Frontend: **Fetch**, **Decode**
 - Backend: **Execute**, **Memory Access**, **Writeback** (Commit)
- Pipeline **hazards** cause a pipeline to **stall** or require a **flush**.
- Pipelines *may* feature **branch prediction**, **out of order execution**, and **speculative execution** features to improve throughput.